

Designing a DevSecOps Toolchain to Support ETL Workflows in Regulated Industries

Daniel Dickerson

October 27, 2025

Contents

1	Introduction	4
1.1	Context and Motivation	4
1.2	Problem Statement	4
1.3	Research Objectives	5
1.3.1	Primary Research Question	5
1.3.2	Sub-Questions	5
1.4	Contributions	5
1.5	Thesis Structure	5
2	Literature Review	7
2.1	Background	7
2.2	Defining Key Concepts	8
2.2.1	DevSecOps	8
2.2.2	ETL	8
2.2.3	Data Governance	9
2.2.4	Auditability	9
2.3	RQ1: ETL Pipelines in Regulated Industries	10
2.4	RQ2: DevSecOps and Governance in the SDLC	10
2.5	RQ3: Comparing the Data Lifecycle and the SDLC	11
2.6	RQ4: Tooling for Auditability and Governance	12
2.7	Synthesis	12
3	Methodology	13
3.1	Research Approach	13
3.2	Design Science Process	13
3.3	Case Study Context	13
3.4	Artifact Design Process	13
3.5	Evaluation Plan	13
3.6	Limitations	14
4	Baseline Pipeline	15
4.1	Overview	15
4.2	Architecture	15
4.3	Functionality	15
4.4	Observed Gaps	15
4.5	Summary	15
5	Proposed Architecture	16
5.1	Design Principles	16
5.2	High-Level Overview	16
5.3	Core Components	16
5.4	Integration with DevSecOps Practices	16
5.5	Expected Outcomes	16
6	Prototype Implementation	17
6.1	Implementation Environment	17

6.2	Toolchain Configuration	17
6.3	Security and Governance Layers	17
6.4	Demonstration	17
6.5	Challenges	17
7	Evaluation	18
7.1	Evaluation Framework	18
7.2	Experimental Setup	18
7.3	Results	18
7.4	Discussion	18
7.5	Threats to Validity	18
7.6	Summary	18
8	Discussion	19
8.1	Summary of Findings	19
8.2	Implications for Practice	19
8.3	Limitations	19
8.4	Future Work	19
9	Conclusion	20
9.1	Restating the Research Question	20
9.2	Key Contributions	20
9.3	Reflections	20
9.4	Closing Remarks	20

Chapter 1

Introduction

1.1 Context and Motivation

In recent years, the value of data has surpassed that of the software systems built to process it [1, 2]. Organisations increasingly treat data as a product, complete with ownership structures, stewardship responsibilities, and lifecycle management frameworks [1]. This shift has coincided with growing regulatory scrutiny across jurisdictions [1]. Frameworks such as APRA’s Prudential Standards [3–6], alongside global instruments such as the EU’s GDPR and the United States’ HIPAA, demand not only the secure handling of information assets but also transparent processes that demonstrate accountability and control [2].

Extract–Transform–Load (ETL) workflows lie at the heart of this transformation, serving as the operational backbone of analytical systems [7, 8]. As Mahmud and Ikbāl [9] note, modern ETL pipelines have evolved into socio-technical infrastructures that sustain the scalability, reliability, and legitimacy of data-driven decision-making. Yet despite the rise of DataOps, many pipelines remain opaque, lacking end-to-end lineage, reproducibility, and verifiable controls [2]. In regulated environments, such opacity undermines compliance and erodes institutional trust.

Software engineering has already faced similar challenges. DevSecOps — the integration of development, security, and operations — has matured into a discipline embedding verification, automation, and governance across the software delivery lifecycle [10–13]. If data is now the product, DevSecOps offers an architectural and cultural foundation for treating data pipelines with the same rigour once reserved for software systems [14]. This research explores that proposition: how DevSecOps principles can be adapted to strengthen auditability and data governance within ETL workflows.

1.2 Problem Statement

While the DataOps movement has improved automation and observability, its primary focus remains operational efficiency rather than regulatory assurance [9, 15]. Existing orchestration frameworks, though powerful, rarely produce immutable audit trails, reproducible execution states, or cryptographically verifiable lineage. Auditability thus emerges as an accidental property rather than a designed one [16, 17]. As a result, organisations face a persistent gap between the technical execution of pipelines and the evidentiary requirements of regulators [3, 4].

This thesis addresses that gap. It investigates how a *DevSecOps-enabled toolchain* can enhance auditability and data governance across the ETL lifecycle. DevSecOps has already institutionalised

security and compliance for software artifacts, but an equivalent framework for data artifacts has yet to emerge. This absence exposes regulated organisations to compliance risk, weakens governance, and impairs their ability to demonstrate trustworthy data management.

1.3 Research Objectives

This research adopts a Design Science methodology [18] to design, implement, and evaluate a DevSecOps-based toolchain that enhances auditability and governance in ETL workflows. The study follows an iterative cycle of artifact design, prototype construction, and evaluation against explicit metrics for lineage completeness, reproducibility, and tamper detection.

1.3.1 Primary Research Question

How can a DevSecOps toolchain be designed to support ETL workflows in regulated industries, enhancing auditability and data governance?

In the process of answering the above question, this paper explores four sub-questions:

1.3.2 Sub-Questions

1. How do ETL workflows manifest in regulated industries, and what auditability and data governance challenges do they face?
2. How do DevSecOps practices enhance auditability and data governance within the software development lifecycle?
3. In what ways do data lifecycle management practices extend or reshape the software development lifecycle when DevSecOps principles are applied to ETL workflows in regulated environments?
4. What categories of tooling exist in DevSecOps and ETL pipelines, and how do their features support auditability and data governance?

1.4 Contributions

This thesis makes three primary contributions. First, it proposes an **architectural model** for a DevSecOps-enabled ETL toolchain, aligning the data lifecycle with secure software delivery practices to improve auditability and governance. Second, it presents a **prototype implementation** that integrates infrastructure as code, automated security scanning, and immutable logging into an operational ETL environment built with Airflow, Spark, and AWS components. Third, it defines an **evaluation framework** for quantifying auditability and governance through traceability, reproducibility, and tamper-resistance metrics. Together, these contributions demonstrate how DevSecOps principles can be embedded in data-centric systems to enhance compliance and trust.

1.5 Thesis Structure

The remainder of this thesis is organised as follows:

- **Chapter 2** reviews literature on ETL pipelines, data governance, and DevSecOps practices.
- **Chapter 3** explains the Design Science methodology, design constraints, and evaluation criteria.
- **Chapter 4** describes the baseline ETL pipeline and identifies auditability and governance gaps.
- **Chapter 5** presents the proposed DevSecOps-integrated architecture.
- **Chapter 6** outlines the prototype implementation and toolchain configuration.
- **Chapter 7** evaluates the system against defined auditability and governance metrics from Chapter 3.
- **Chapter 8** discusses implications, limitations, and avenues for future research.
- **Chapter 9** concludes with reflections on how DevSecOps can enable trustworthy data governance in regulated environments.

Chapter 2

Literature Review

This chapter reviews the body of knowledge relevant to the design of ETL pipelines in regulated industries and the application of DevSecOps principles. It begins by outlining the context of ETL and governance, defines the key concepts central to this research, and then examines the literature associated with each research question.

2.1 Background

Data pipelines underpin modern analytical systems, forming the connective infrastructure between data sources and analytic platforms [14]. They transform raw data into governed, traceable, and auditable outputs used in applications such as fraud detection, market analysis, compliance reporting, and metadata processing.

Although the extract–transform–load (ETL) process appears simple in theory, it is in practice complex, resource-intensive, and often the most time-consuming element of data warehousing [19]. ETL pipelines act as the “eyes of the system,” feeding reliable data downstream; when lineage is lost or visibility fails, the reliability of the entire process is compromised [8]. Vayyala [8] emphasises that maintaining end-to-end lineage across transformations is vital for sustaining trust in analytics-driven decisions.

In regulated industries, compliance depends as much on how data is processed as on the outcomes produced. Frameworks such as GDPR, SOX, and APRA’s CPS standards require secure, ethical, and well-documented data handling [1, 3–6]. These frameworks place accountability, ownership, and audit trails at the centre of compliance culture.

Auditability, in this context, is the ability to demonstrate compliance through reproducibility, lineage, and tamper-resistant logs [2]. Trustworthy analytics rely not only on integrity of data but also on immutable evidence of correct process execution. Governance provides the organisational scaffolding for these assurances—linking policy to technical enforcement through defined roles, procedures, and controls [1, 2]. Failures in either dimension expose organisations to legal, financial, and reputational risk [1].

DevOps emerged to accelerate software delivery through automation, continuous integration and deployment (CI/CD), and rapid feedback cycles [13]. DevSecOps extends this model by embedding security and compliance across the software development lifecycle. Practices such as automated testing, compliance-as-code, artifact signing, and immutable logging produce verifiable artifacts that can be inspected by regulators [10, 11, 13]. This convergence bridges engineering and governance concerns, establishing compliance as a built-in property of the delivery process.

As data itself becomes a primary strategic asset, surpassing the value of the systems that process it [1, 2], the mechanisms of DevSecOps must be extended to the data pipelines that sustain organisational decision-making.

2.2 Defining Key Concepts

The following subsections define the key concepts underpinning this research: DevSecOps, ETL, data governance, and auditability. Each definition draws from existing literature and sets the theoretical foundation for later analysis.

2.2.1 DevSecOps

DevSecOps is the proactive and continuous enforcement of security and compliance requirements throughout the software development lifecycle (SDLC) [10]. Unlike traditional models that treat security as an external stage or afterthought, DevSecOps embeds it within every phase of the lifecycle [12]. Compliance becomes the default state, and deviations from defined standards must be explicitly justified [11].

A defining characteristic of DevSecOps is its reliance on automation to enforce non-functional requirements such as auditability and governance. Through automated testing, provisioning, monitoring, and policy enforcement, systems produce immutable evidence—signed artifacts, logs, and metrics—that regulators and auditors can independently assess [10, 20]. This automated assurance bridges the concerns of development teams and governance stakeholders [12].

By combining cultural expectations of shared responsibility with technical mechanisms for verification, DevSecOps establishes a delivery model in which security and compliance are continuous, measurable, and intrinsic—rather than externally imposed.

2.2.2 ETL

Extract–Transform–Load (ETL) is a structured process that ingests, transforms, and persists data into target repositories such as data warehouses, lakes, or file stores [19]. It traditionally comprises three stages: extraction, where data is sourced from heterogeneous systems; transformation, where it is cleansed, standardised, integrated, and reshaped into coherent semantic structures; and loading, where transformed data is stored for downstream analysis [19]. Kimball and Caserta describe ETL as both the most resource-intensive and most critical link between operational data and analytical use.

Modern practice extends beyond this classical batch model. Approaches such as extract–load–transform (ELT) and hybrid streaming–batch architectures leverage distributed and cloud-native infrastructure [9]. ETL now encompasses not only data integration and quality but also governance and compliance. In regulated industries, pipelines must guarantee lineage, reproducibility, and auditability in addition to performance and scalability [9].

ETL is thus best understood as a flexible socio-technical process. It concludes when data is made persistent and reusable but intersects with broader lifecycle concerns such as orchestration, monitoring, and policy enforcement. In this sense, ETL pipelines underpin both analytical scalability and institutional trust [9].

2.2.3 Data Governance

Data governance provides the strategic framework that defines how data is collected, managed, and protected across an organisation. It establishes the policies, roles, standards, and processes that ensure information assets remain accurate, consistent, secure, and compliant with internal and external regulations [1, 2]. Nai, Rifai, and Sadiq [1] describe governance as a structured framework that ensures the availability, integrity, usability, and security of data through stewardship and accountability roles.

Vayyala [2] expands this understanding by positioning governance as the foundation of reliable data ecosystems—linking policy to technical enforcement through automation, lineage tracking, and immutable audit logs. In practice, governance institutionalises compliance and accountability, transforming data into a verifiable strategic asset that can withstand regulatory scrutiny.

2.2.4 Auditability

Auditability is the designed capability of a system to allow its processes, controls, and outcomes to be observed and verified by stakeholders. Weigand and Elsas [17] define auditability through five interdependent requirements:

- **Normativity:** a clear regulatory or contractual framework that governs operations.
- **Accountability:** verifiable statements linking actions to responsible agents and governing norms.
- **Referentiality Infrastructure:** records that connect operations to their original points of recording.
- **Reliability:** faithful and tamper-resistant representation of events.
- **Verifiability:** the ability of external parties to independently test or validate claims.

Although these principles originate in organisational and financial control, they can be transferred to information systems. Extending this theoretical foundation, Wang et al. [16] demonstrate how auditability can be operationalised through third-party verification, cryptographic proofs of integrity, and support for dynamic data operations. Their work illustrates that auditability is not merely an organisational goal but a property that can be engineered into data management infrastructures.

Applied to ETL pipelines and DevSecOps practices, these five dimensions map directly to regulated-industry concerns:

- **Normativity:** embedding compliance and data-protection policies within workflow execution.
- **Accountability:** linking each transformation or deployment to authenticated agents or automated processes.
- **Referentiality Infrastructure:** ensuring full lineage tracking across extraction, transformation, and loading.
- **Reliability:** maintaining immutable logs, signed artifacts, and cryptographic hashes.
- **Verifiability:** enabling reproducibility tests and independent audits via CI/CD-integrated checks.

Together, these adapted requirements frame auditability as a designed and engineered property of ETL systems. Every data operation and system change must be traceable, reproducible, and independently verifiable against regulatory expectations.

2.3 RQ1: ETL Pipelines in Regulated Industries

ETL pipelines in regulated industries operate within a constant tension between agility and accountability. They are expected to deliver analytical value with speed and scale, yet must simultaneously produce verifiable evidence that every data movement complies with institutional and regulatory constraints. The modern pipeline has therefore evolved into a socio-technical infrastructure—one that mediates trust between engineers, auditors, and regulators [8, 9]. Its success cannot be judged solely by throughput or latency; rather, by its ability to maintain lineage, transparency, and compliance under continuous change.

Financial, healthcare, and critical-infrastructure systems exemplify this dual demand. Frameworks such as APRA’s CPS 230, CPS 234, and CPS 510 [3–5] institutionalise separation of duties and controlled promotion of change, translating ethical responsibility into technical procedure. These standards do not exist apart from engineering practice—they shape it—embedding legal accountability into operational design. Governance scholarship reaches similar conclusions, emphasising that immutable logging, role-based accountability, and retention aligned with compliance lifecycles form the practical machinery of institutional trust [1, 2, 7]. Cloud-native orchestrators such as Airflow, AWS Glue, and Azure Data Factory provide elasticity and automation, but they operate under the shadow of these mandates. Their convenience is constrained by the need to preserve traceability and access control consistent with CPG 235 [6]. In this context, orchestration itself becomes a form of governance: a choreography of tasks and dependencies that leaves a verifiable trail of execution [21].

Over time, this operational discipline has turned the pipeline from a means of data transport into a living archive of compliance. Continuous validation, versioning, and policy enforcement integrate audit logic directly into the runtime environment [2]. Metadata management forms the connective tissue between policy and process, ensuring that provenance, schema evolution, and transformation logic can be reconstructed and defended [7]. Yet even within mature systems, full auditability remains elusive. Toolchain heterogeneity fragments lineage across layers of orchestration, computation, and storage. Business logic embedded in SQL or user-defined functions escapes formal tracing, while schema drift and inconsistent metadata propagation corrode reliability [9]. Few frameworks create immutable, cryptographically-verifiable artefacts by default [7, 17]. What is missing is not information, but integrity—the ability to prove that recorded operations are both complete and untampered. The fragility of this evidentiary chain provides the practical motivation for integrating DevSecOps assurance into ETL environments, turning verification from an external audit to an intrinsic system property.

2.4 RQ2: DevSecOps and Governance in the SDLC

Where ETL pipelines reveal the symptoms of audit fragility, the software development lifecycle (SDLC) offers a model for its remedy. The SDLC formalises change as a controlled process, translating intent into artefact through a sequence of verifiable stages. Standards such as ISO/IEC/IEEE 12207 and 15288 codify these stages into information items—plans, specifications, procedures, and reports—that render software creation traceable and accountable [22, 23]. Governance is therefore not an afterthought but a structural feature of disciplined development.

DevSecOps intensifies this structure by embedding assurance, security, and compliance mechanisms into each phase of delivery [10, 24]. It reframes the pipeline not merely as a path to deployment but as a generator of evidence. Each build, test, and deployment event produces signed artefacts, immutable logs, and compliance manifests—machine-verifiable records that bind design to behaviour [10, 12, 17, 25]. These artefacts form a distributed chain of trust, updated with every iteration. The system thus audits itself as it operates, transforming assurance from a scheduled inspection to a continuous state.

The philosophy extends through automation patterns such as infrastructure-as-code, continuous integration and delivery, and policy-as-code. Automated testing, dependency scanning, and approval gates make verification habitual rather than exceptional [12, 23, 26]. NIST’s Secure Software Development Framework (SSDF) [24] codifies these expectations: traceable metadata, provenance records, and policy enforcement are no longer optional but integral lifecycle artefacts. When embedded in CI/CD toolchains, these mechanisms sustain persistent audit readiness and measurable compliance [6, 17].

The growing complexity of digital supply chains has made this form of assurance indispensable. Expanding attack surfaces, opaque third-party dependencies, and volatile cloud environments require constant verification of integrity and origin [24, 26]. DevSecOps adapts by introducing autonomous scanning, provenance analysis, and even AI-assisted policy reasoning to detect deviation as it occurs [27, 28]. Requirement traceability links regulatory intent to runtime artefacts [22, 25], ensuring that what is built, what is deployed, and what is audited all refer to the same authoritative chain of evidence [13]. The SDLC thereby evolves into an operational architecture of continuous assurance—a framework where compliance is produced, not declared.

2.5 RQ3: Comparing the Data Lifecycle and the SDLC

The data lifecycle (DLC) mirrors the SDLC’s ambition for control but lacks its formal codification. Wing [29] describes the DLC as a dynamic abstraction, moulded by institutional context rather than dictated by a single methodology. Despite the absence of a universal model, most frameworks converge on a shared aim: to maintain reliability, accessibility, and verifiability across the data’s lifespan [30, 31]. The contrast is instructive. The SDLC governs transformation and release; the DLC governs persistence and stewardship. One manages the creation of systems, the other their informational memory. Both, however, rely on artefacts that can sustain trust through time.

DAMA-DMBOK [32] formalises this idea by organising data governance around defined roles, metadata catalogues, lineage reports, and quality assessments that together form an auditable chain of evidence. These mirror the artefacts enumerated in ISO/IEC/IEEE 15289 [22], suggesting that the epistemic logic of software assurance—control through documentation—translates naturally into data management. DevSecOps provides the operational mechanism for this translation. By embedding continuous verification and security controls across both lifecycles [10, 24], DevSecOps aligns the treatment of data artefacts with that of software artefacts: each becomes a controlled, versioned, and verifiable object.

This alignment is not merely procedural but conceptual. Within a unified lifecycle, lineage manifests, schema definitions, and transformation scripts stand alongside code repositories and configuration files as equal participants in compliance. Encryption, secrets management, and role-based access controls produce logs and proofs of activity that serve as the raw material for regulatory assurance [14, 33]. Agile governance models add an iterative layer, moving validation and data-quality checks upstream to the earliest stages of development [7, 8, 34]. These frameworks echo the stewardship and control expectations embedded in DAMA-DMBOK and in APRA’s prudential standards [3–6], transforming compliance from a retrospective audit to a property continuously generated by the system itself.

When the SDLC and DLC converge, they form a continuous system of evidence. The information items of ISO/IEC/IEEE 15289—plans, specifications, procedures, and reports—extend naturally to include data-lineage documentation, schema-evolution logs, and validation results. Automation frameworks such as infrastructure-as-code and policy-as-code operationalise these records [12, 13, 35], ensuring that reproducibility, tamper detection, and governance share the same delivery pipelines as application code. In this synthesis, the distinction between developing software and managing data collapses into a single assurance practice grounded in verifiable process.

Ultimately, the integration of these lifecycles reframes auditability as both a technical and organisational design principle. Governance, provenance, and verification cease to be external constraints; they become

generative forces that shape how systems evolve. In regulated ETL workflows, every transformation of code or data leaves a signed, immutable trace—an artefact that documents not only what occurred, but why it was permitted to occur. This capacity to bind intent, execution, and evidence within a single architectural rhythm marks the threshold where compliance becomes indistinguishable from engineering itself.

2.6 RQ4: Tooling for Auditability and Governance

2.7 Synthesis

Chapter 3

Methodology

3.1 Research Approach

Explain the overall design science methodology used. Justify its selection for solving an engineering problem through artifact design.

3.2 Design Science Process

Describe each stage:

- **Problem identification:** Auditability and governance gaps in ETL.
- **Artifact design:** A DevSecOps toolchain architecture.
- **Demonstration:** Implementing the prototype.
- **Evaluation:** Measuring lineage, reproducibility, and tamper detection.

3.3 Case Study Context

Briefly describe the regulated domain (e.g., financial systems, SEC filings dataset). Justify its representativeness and constraints.

3.4 Artifact Design Process

Detail the iterative development process—tools, versioning, and validation loops.

3.5 Evaluation Plan

Summarize how you'll measure auditability, reproducibility, and tamper resistance.

3.6 Limitations

Acknowledge what is out of scope or constrained (e.g., scalability, dataset variety).

Chapter 4

Baseline Pipeline

4.1 Overview

Describe the initial ETL setup (Airflow + Spark + S3, deployed via Docker). Explain its purpose as a comparison point.

4.2 Architecture

Insert a diagram showing the baseline system structure. Highlight key components and data flow.

4.3 Functionality

Detail how data moves through extraction, transformation, and loading.

4.4 Observed Gaps

Discuss deficiencies in:

- Lineage tracking
- Reproducibility
- Governance and security controls

4.5 Summary

Conclude with why the baseline cannot meet auditability expectations.

Chapter 5

Proposed Architecture

5.1 Design Principles

Outline the guiding principles of the proposed DevSecOps-integrated ETL system: automation, immutability, traceability, reproducibility.

5.2 High-Level Overview

Provide an architectural diagram comparing baseline and proposed design.

5.3 Core Components

Explain each layer:

- **Infrastructure as Code:** Terraform for provisioning.
- **Orchestration:** Airflow for controlled execution.
- **Security:** CI/CD checks, artifact signing, SAST/SCA.
- **Governance:** Immutable audit logs, metadata tracking.

5.4 Integration with DevSecOps Practices

Map the proposed pipeline to the DevSecOps lifecycle (plan → code → build → test → release → monitor).

5.5 Expected Outcomes

Summarize the architectural advantages for auditability and compliance.

Chapter 6

Prototype Implementation

6.1 Implementation Environment

List the environment setup: AWS EMR, EC2, S3, Docker, GitHub Actions, Terraform.

6.2 Toolchain Configuration

Show how tools interconnect (e.g., Airflow triggers Spark, CI/CD automates IaC).

6.3 Security and Governance Layers

Describe:

- Hash signing of artifacts
- Immutable logging mechanisms
- Versioned configuration and audit history

6.4 Demonstration

Summarize execution flow with example runs and screenshots.

6.5 Challenges

Briefly discuss debugging, integration issues, and trade-offs during build.

Chapter 7

Evaluation

7.1 Evaluation Framework

Define how the artifact is assessed — e.g., lineage completeness, reproducibility, tamper detection.

7.2 Experimental Setup

Describe datasets, test environments, and CI/CD validation routines.

7.3 Results

Present tables, metrics, and visualizations of outcomes.

7.4 Discussion

Interpret results in context: - How did the toolchain improve governance? - What measurable gains were observed?

7.5 Threats to Validity

Discuss internal and external validity considerations.

7.6 Summary

Wrap up the findings and prepare for the discussion/conclusion chapters.

Chapter 8

Discussion

8.1 Summary of Findings

Recap the main evaluation results and architectural improvements.

8.2 Implications for Practice

Explain how this approach could generalize to other regulated environments.

8.3 Limitations

Note scalability, data type constraints, or untested regulatory frameworks.

8.4 Future Work

Propose enhancements—e.g., integrating ML pipelines, policy-as-code frameworks, or extended logging.

Chapter 9

Conclusion

9.1 Restating the Research Question

Reiterate the main question and summarize how your design addressed it.

9.2 Key Contributions

List concrete deliverables (architecture, prototype, metrics, evaluation).

9.3 Reflections

Discuss broader lessons about DevSecOps integration into data pipelines.

9.4 Closing Remarks

Tie back to the motivation and note the continuing relevance for regulated industries.

Bibliography

- [1] S. Nai, A. Rifai, and A. Sadiq, “Data Governance, Key Insights, Strategic Challenges, and Future Imperatives;,” in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 1–16. [Online]. Available: <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch001>
- [2] R. Vayyala, “Ensuring Data Quality and Integrity in Modern Enterprises;,” in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 17–46. [Online]. Available: <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch002>
- [3] Australian Prudential Regulation Authority, “Prudential Standard CPS 234: Information Security,” Canberra, ACT, Jul. 2019. [Online]. Available: https://www.apra.gov.au/sites/default/files/cps_234_july_2019_for_public_release.pdf
- [4] —, “Prudential Standard CPS 230: Operational Risk Management,” Canberra, ACT, Jul. 2025. [Online]. Available: <https://www.apra.gov.au/sites/default/files/2025-07/Prudential%20Standard%20CPS%20230%20Operational%20Risk%20Management%20-%20clean.pdf>
- [5] —, “Prudential Standard CPS 510: Governance,” Canberra, ACT, Jul. 2023. [Online]. Available: https://www.apra.gov.au/sites/default/files/prudential_standard_cps_510_governance.pdf
- [6] —, “Prudential Practice Guide CPG 235: Managing Data Risk,” Australian Prudential Regulation Authority (APRA), Canberra, ACT, Prudential Practice Guide CPG 235, Sep. 2013. [Online]. Available: https://www.apra.gov.au/sites/default/files/Prudential-Practice-Guide-CPG-235-Managing-Data-Risk_1.pdf
- [7] R. Vayyala, “Metadata Management and Its Role in Data Governance;,” in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 47–72. [Online]. Available: <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch003>
- [8] —, “Data Lineage: Tracing Data Across Systems,” in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 73–98. [Online]. Available: <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch004>
- [9] D. Mahmud and M. Z. Ikbali, “THE ROLE OF ETL (EXTRACT-TRANSFORM-LOAD) PIPELINES IN SCALABLE BUSINESS INTELLIGENCE: A COMPARATIVE STUDY OF DATA INTEGRATION TOOLS,” *ASRC Procedia: Global Perspectives in Science and Scholarship*, vol. 2, no. 1, pp. 89–121, Apr. 2022. [Online]. Available: <https://global.asrcconference.com/index.php/asrc/article/view/29>
- [10] M. Anisetti, N. Bena, F. Berto, and G. Jeon, “A DevSecOps-based Assurance Process for Big Data Analytics,” in *2022 IEEE International Conference on Web Services (ICWS)*, Jul. 2022, pp. 1–10. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9885738>
- [11] C. Castellanos, C. A. Varela, and D. Correal, “ACCORDANT: A domain specific-model and DevOps approach for big data analytics architectures,” *Journal of Systems and Software*, vol.

172, p. 110869, Feb. 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0164121220302594>

- [12] M. Anisetti, C. A. Ardagna, E. Damiani, and F. Gaudenzi, “A Semi-Automatic and Trustworthy Scheme for Continuous Cloud Service Certification,” *IEEE Transactions on Services Computing*, vol. 13, no. 1, pp. 30–43, Jan. 2020. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/7831357/authors>
- [13] P. Pandey and A. Patel, “Integrating Security in Cloud-Native Development: A DevSecOps Approach to Resilient Software Systems,” in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 169–196. [Online]. Available: <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch009>
- [14] K. Soussane, B. E. Elbaghazaoui, and M. Amnai, “Integrating Security in DataOps: A Framework for Securing Data Pipelines in Real-time Analytics,” in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 197–214. [Online]. Available: <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch010>
- [15] A. R. Munappy, D. I. Mattos, J. Bosch, H. H. Olsson, and A. Dakkak, “From Ad-Hoc Data Analytics to DataOps,” in *Proceedings of the International Conference on Software and System Processes*, ser. ICSSP ’20. New York, NY, USA: Association for Computing Machinery, 2020, pp. 165–174, event-place: Seoul, Republic of Korea. [Online]. Available: <https://doi.org/10.1145/3379177.3388909>
- [16] Q. Wang, C. Wang, K. Ren, W. Lou, and J. Li, “Enabling Public Auditability and Data Dynamics for Storage Security in Cloud Computing,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 22, no. 5, pp. 847–859, May 2011. [Online]. Available: <https://ieeexplore.ieee.org/document/5611497>
- [17] H. Weigand and P. Elsas, “Auditability as a Design Problem,” in *2019 IEEE 21st Conference on Business Informatics (CBI)*, vol. 01, Jul. 2019, pp. 276–285, iSSN: 2378-1971. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8808092>
- [18] A. Hevner, A. R. S. March, S. T. Park, J. Park, Ram, and Sudha, “Design Science in Information Systems Research,” *Management Information Systems Quarterly*, vol. 28, pp. 75–, Mar. 2004.
- [19] R. Kimball and J. Caserta, *The Data Warehouse ETL Toolkit: Practical Techniques for Extracting, Cleaning, Conforming, and Delivering Data*. Wiley, 2004, iISBN: 9780764567575. [Online]. Available: <https://www.oreilly.com/library/view/the-data-warehouse/9780764567575/>
- [20] Y. Zhou, Y. Yu, and B. Ding, “Towards MLOps: A Case Study of ML Pipeline Platform,” in *2020 International Conference on Artificial Intelligence and Computer Engineering (ICAICE)*, Oct. 2020, pp. 494–500. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9361315>
- [21] A. Pogiatis and G. Samakovitis, “An Event-Driven Serverless ETL Pipeline on AWS,” *Applied Sciences*, vol. 11, no. 1, p. 191, Jan. 2021, publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: <https://www.mdpi.com/2076-3417/11/1/191>
- [22] “ISO/IEC/IEEE International Standard – Systems and software engineering - Content of life-cycle information items (documentation),” Jul. 2019. [Online]. Available: <https://ieeexplore.ieee.org/document/8767110>
- [23] R. Hernández, B. Moros, and J. Nicolás, “Requirements management in DevOps environments: a multivocal mapping study,” *Requirements Engineering*, vol. 28, no. 3, pp. 317–346, Sep. 2023. [Online]. Available: <https://doi.org/10.1007/s00766-023-00396-w>
- [24] M. Souppaya, K. Scarfone, and D. Dodson, “Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities,” National Institute of Standards and Technology, Tech. Rep. NIST Special Publication (SP) 800-218, Feb. 2022. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/218/final>

- [25] T. Chen and H. Suo, “Design and Practice of Security Architecture via DevSecOps Technology,” in *2022 IEEE 13th International Conference on Software Engineering and Service Science (ICSESS)*, Oct. 2022, pp. 310–313, iSSN: 2327-0594. [Online]. Available: <https://ieeexplore.ieee.org/document/9930212>
- [26] J. Li and H. Li, “Evolution of Application Security based on OWASP Top 10 and CWE/SANS Top 25 with Predictions for the 2025 OWASP Top 10,” in *2025 International Conference on Inventive Computation Technologies (ICICT)*, Apr. 2025, pp. 1178–1183, iSSN: 2767-7788. [Online]. Available: <https://ieeexplore.ieee.org/document/11004742>
- [27] A. Mittal and V. Venkatesan, “Practical Integration of Large Language Models into Enterprise CI/CD Pipelines for Security Policy Validation: An Industry-Focused Evaluation,” in *2025 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, Jul. 2025, pp. 197–203, iSSN: 2642-6587. [Online]. Available: <https://ieeexplore.ieee.org/document/11126149>
- [28] B. Achuthan and M. A. Alimohideen, “Shifting Gears: Integrating Security Audits into Automotive DevSecOps,” in *2024 International Conference on Vehicular Technology and Transportation Systems (ICVTTS)*, vol. 1, Sep. 2024, pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/10763937>
- [29] J. M. Wing, “The Data Life Cycle,” *Harvard Data Science Review*, vol. 1, no. 1, Jul. 2019, publisher: The MIT Press. [Online]. Available: <https://hdr.mitpress.mit.edu/pub/577rq08d/release/4>
- [30] A. Ball, *Review of Data Management Lifecycle Models*. Bath, UK: University of Bath, Feb. 2012.
- [31] W. C. Lenhardt, S. Ahalt, B. Blanton, L. Christopherson, and R. Idaszak, “Data Management Lifecycle and Software Lifecycle Management in the Context of Conducting Science | Journal of Open Research Software,” Feb. 2014. [Online]. Available: <https://openresearchsoftware.metajnl.com/articles/10.5334/jors.ax>
- [32] P. Cupoli, S. Earley, and D. Henderson, “DAMA-DMBOK2 Framework.”
- [33] A. Patel and P. Pandey, “Safeguarding Sensitive Data in the DevSecOps Pipelines: Strategies, Tools, and Best Practices for Modern Software Development,” in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 215–240. [Online]. Available: <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch011>
- [34] B. O. Kose, “Agility Meets Compliance: The Convergence of Data Governance and DevSecOps,” in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 131–154. [Online]. Available: <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch007>
- [35] H. Myrbakken and R. Colomo-Palacios, “DevSecOps: A Multivocal Literature Review,” in *Software Process Improvement and Capability Determination*, A. Mas, A. Mesquida, R. V. O’Connor, T. Rout, and A. Dorling, Eds. Cham: Springer International Publishing, 2017, pp. 17–29.